■ 2.6.15 Special Topic: Writing Output to Files and Pipes

The operators `>>` and `>>>` are the standard *Mathematica* functions for writing output to files and pipes. There are some extensions to these functions which are often used.

expr `>>` *file*	write an expression to a file, in input format
expr `>>>` *file*	append an expression to a file, in input format
`OutputForm[`*expr*`] >>` *file*	write an expression to a file, in output format
`Display["`*file*`",` *graphics*`]`	write graphics to a file, in PostScript form
`Save["`*file*`",` s_1, s_2, ...`]`	save the complete definitions of the symbols s_i in a file
`Definition[`*s*`] >>` *file*	write the definition of *s* to a file
`FullDefinition[`*s*`] >>` *file*	write the complete definition of *s* to a file, including definitions of any objects that *s* depends on

Writing output to files.

All functions for writing output to files can also be used to send output through pipes to external commands. You can specify a pipe simply by giving an external command, prefaced by `!`, in place of a file name.

This writes the expression `a/b` to the file `tmp.m`.

```
In[1]:= a/b >> tmp.m
```

The operator `>>>` appends to a file.

```
In[2]:= c/d >>> tmp.m
```

The file `tmp.m` now contains both expressions, given in input form.

```
In[3]:= !!tmp.m

a/b
c/d
```

When you use `>>` and `>>>` to write expressions to files, the expressions are usually given in input format, so that they can be read back into *Mathematica*. Sometimes you may want to save expressions in other formats.

By specifying an explicit output form, you can override the default choice of input format.

```
In[3]:= OutputForm[x^2 + y^2] >> tmp.m
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; http://www.wolfram.com/; 1-800-441-6284.

The expression was written to the file in output format.

```
In[4]:= !!tmp.m
 2    2
x  + y
```

This writes out the expression in FORTRAN form.

```
In[4]:= FortranForm[x^2 + y^2] >> tmp.m
```

One of the most common reasons for using files is to save definitions of *Mathematica* objects, to be able to read them in again in a subsequent *Mathematica* session. The operators `>>` and `>>>` allow you to save *Mathematica* expressions in files. You can use the function `Save` to save complete definitions of *Mathematica* objects, in a form suitable for execution in subsequent *Mathematica* sessions.

This assigns a value to the symbol `a`.

```
In[5]:= a = 2 - x^2
                2
Out[5]= 2 - x
```

You can use `Save` to write the definition of `a` to a file.

```
In[6]:= Save["afile.m", a]
```

Here is the definition of `a` that was saved in the file.

```
In[7]:= !!afile.m
a = 2 - x^2
```

When you define a new object in *Mathematica*, your definition will often depend on other objects that you defined before. If you are going to be able to reconstruct the definition of your new object in a subsequent *Mathematica* session, it is important that you store not only its own definition, but also the definitions of other objects on which it depends. The function `Save` looks through the definitions of the objects you ask it to save, and automatically also saves all definitions of other objects on which these depend. As a result, reading the output generated by `Save` back into a new *Mathematica* session should set up the definitions of your objects exactly as you had them before.

This defines a function `f` which depends on the symbol `a` defined above.

```
In[7]:= f[z_] := a^2 - 2
```

This saves the complete definition of `f` in a file.

```
In[8]:= Save["ffile.m", f]
```

The file contains not only the definition of `f` itself, but also the definition of the symbol `a` on which `f` depends.

```
In[9]:= !!ffile.m
f/: f[z_] := a^2 - 2
a = 2 - x^2
```

The function `Save` makes use of the output forms `Definition` and `FullDefinition`, which print as definitions of *Mathematica* symbols. In some cases, you may find it convenient to use these output forms directly.

The output form `Definition[`*f*`]` prints as the sequence of definitions that have been made for *f*.

```
In[9]:= Definition[f]

                      2
Out[9]= f/: f[z_] := a  - 2
```

`FullDefinition[`*f*`]` includes definitions of the objects on which *f* depends.

```
In[10]:= FullDefinition[f]

                       2
Out[10]= f/: f[z_] := a  - 2

                 2
         a = 2 - x
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; http://www.wolfram.com/; 1-800-441-6284.